Software Testing, Quality Assurance & Maintenance—Lecture 17

Patrick Lam
University of Waterloo

March 9, 2026

Part I

Property testing with Bolero; extension to Kani

Last time: Proptest vs proof harness

Let's compare.

```
#[cfg(kani)]
#[kani::proof]
fn check_estimate_size() {
    let x: u32 = kani::any();
    estimate_size(x);
}
```

```
#[test]
fn doesnt_crash(x: u32) {
    estimate_size(x);
}
```

...still pretty similar!

Bolero

meta-framework for property-testing:
integrates fuzzing engines & also Kani.

Example: Property testing with Bolero

Source: https://model-checking.github.io/kani-verifier-blog/2022/10/27/using-kani-with-the-bolero-property-testing-framework.html.

```
fn update_account_balance(current_balance: i32,
                          amount: i32) -> i32 {
    return current_balance + amount;
}
```

(Blog post code is not executable, ours sort of is.)

Concrete test

```
fn balance_upd_is_correct(current_balance: i32,
    amount: i32, new_balance: i32) -> bool {
  // also: checks other properties
  return current_balance + amount == new_balance; }

#[test]
fn test_update_account_balance() {
  let current_balance = 5; let amount = 2;
  let nb = update_account_balance(current_balance,
                                  amount);
  assert!(balance_upd_is_correct(current_balance,
                                 amount, nb));
}
```

We can `cargo test` this, and it succeeds.

DIY random testing

```rust
#[test]
fn test_update_account_balance_random() {
 let current_balance = rand::random();
 let amount = rand::random();
 let result = update_account_balance
                 (current_balance, amount);
 assert!(balance_update_is_correct
                 (current_balance, amount, result));
}
```

But we've seen better: coverage-guided fuzzing.

Fundamental Theorem of Software Engineering

"We can solve any problem by introducing an extra level of indirection."

(per Wikipedia, attributed to David J. Wheeler)

Here: Bolero abstracts away from specific fuzzing engine.

Bolero at work

```
#[test]
#[cfg_attr(kani, kani::proof)]
fn test_update_account_balance_bolero() {
    bolero::check!().with_type::<(i32, i32)>().cloned(
        let result = update_account_balance(current_ba
        assert!(balance_update_is_correct(current_bala
    });
}
```

I can definitely run this with libfuzzer and Kani.
For technical reasons, AFL doesn't work on my system.
I don't have honggfuzz installed.

Libfuzzer under Bolero

```
$ cargo bolero test test_update_account_balance_bolero --engine libfuzzer
```

tells us that our code fails on overflow:

```
======================== Test Failure ========================

Input:
(
    369098752,
    1778384896,
)

Error:
panicked at src/main.rs:6:12:
attempt to add with overflow
note: run with `RUST_BACKTRACE=1` environment variable to display a backtr
```

Kani under Bolero

```
$ cargo bolero test test_update_account_balance_bolero --engine kani
[...]
SUMMARY:
 ** 1 of 238 failed (1 unreachable)

 ** 1 of 1 cover properties satisfied

Failed Checks: attempt to add with overflow
 File: "src/main.rs", line 6, in update_account_balance

VERIFICATION:- FAILED
Verification Time: 0.16379055s
```

Another Bolero example: nth-rev

(Incorrectly) return n^{th}-from-the-end element as `Some(x)`.

```
fn nth_rev(arr: &[i32], index: usize) -> Option<i32> {
    if index < arr.len() {
        let rev_index = arr.len() - index /* - 1 */;
        return Some(arr[rev_index]);
    }
    None
}
```

Bolero harness

Check that we get `Some()` from each element.

```
const N: usize = 5; // bound for BMC

#[test]
#[cfg_attr(kani, kani::proof)] // if Kani, mark as proof ha
fn check_rev() {
    bolero::check!()
        .with_type::<([i32; N], usize)>()
        .cloned()
        .for_each(|(arr, index)| {
            let x = nth_rev(&arr, index);
            if index < arr.len() {
                assert!(x.is_some());
            }
        });
}
```

libfuzzer finds the bug

```
$ cargo bolero test check_rev --engine libfuzzer
[...]
test failed; shrinking input...

======================== Test Failure ========================

Input:
(
    [
        0,
        0,
        0,
        0,
        0,
    ],
    0,
)

Error:
panicked at src/main.rs:8:21:
index out of bounds: the len is 5 but the index is 5
```

Kani finds the bug

```
$ cargo bolero test check_rev --engine kani
[...]
SUMMARY:
 ** 1 of 698 failed (8 unreachable)

 ** 1 of 1 cover properties satisfied

Failed Checks: index out of bounds: the length is less than or equal
 File: "src/main.rs", line 8, in nth_rev

VERIFICATION:- FAILED
Verification Time: 1.0711443s
```

After fixing, Kani succeeds

```
SUMMARY:
 ** 0 of 699 failed (8 unreachable)

 ** 1 of 1 cover properties satisfied


VERIFICATION:- SUCCESSFUL
Verification Time: 1.0993001s
```

libfuzzer also doesn't find any bugs on an overnight run.

Why both Kani and libfuzzer

My take:

- Kani and libfuzzer succeeded and failed on same programs.
- This is not true in general.

Perhaps fuzzing doesn't find anything,
but Kani does.

Or, Kani might not find an issue,
but fuzzing does.

Outputs from Kani and libfuzzer

Both fuzzing and Kani might give a test case;
it's real, and you can fix it.

Unwinding assertion failure: must investigate more.
Kani success: your program is correct modulo the bound,
assuming everything else on the system correct.

Fuzzing says nothing: don't really know that your
program is correct, but have some warm and fuzzy
feelings.

Under the hood: Bolero & Kani

Bolero uses `kani::any()` to get a symbolic value,
`kani::assume()` to restrict the range, e.g.

```
bolero::check!().with_generator(5..37)
    .for_each( // snip... );
```

becomes

```
let value = kani::any();
kani::assume((5..37).contains(&value));
value
```

Part II

Kani (BMC) to Dafny (auto-active program verification)

About Dafny

Dafny: automatically verify annotated programs.

In SE212, manually (with George's help) prove programs.

Dafny does the same, but mostly automatically.

Terminology: "auto-active" (versus interactive).

Kani vs Dafny: Rust Rectangle example

```rust
#[derive(Debug, Copy, Clone)]
struct Rectangle {
    width: u64, height: u64,
}

impl Rectangle {
    fn can_hold(&self, other: &Rectangle) -> bool {
        self.width > other.width &&
        self.height > other.height
    }

    fn stretch(&self, factor: u64) -> Option<Self> {
        let w = self.width.checked_mul(factor)?;
        let h = self.height.checked_mul(factor)?;
        Some(Rectangle {
            width: w,
            height: h,
        })
    }
}
```

Dafny Rectangle

```
class Rectangle {
    var width: int
    var height: int
}

predicate can_hold(self: Rectangle, other: Rectangle)
    reads self, other
{
    self.width > other.width ∧ self.height > other.height
}

method stretch(self: Rectangle, factor: nat)
        returns (rv : Rectangle) {
    rv := new Rectangle;
    rv.width := self.width * factor;
    rv.height := self.height * factor;
    return rv;
}
```

Rust concrete test case

Just an average test case.

```rust
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn stretched_rectangle_can_hold_original() {
        let original = Rectangle {
            width: 8,
            height: 7,
        };
        let factor = 2;
        let larger = original.stretch(factor);
        assert!(larger.unwrap().can_hold(&original));
    }
}
```

Dafny concrete test case

Just an average test case (in Dafny).

```
method {:test} TestStretchedRectangleCanHoldOriginal() {
    var original := new Rectangle;
    original.width := 8; original.height := 7;
    var factor := 2;
    var larger := stretch(original, factor);
    expect can_hold(larger, original);
}
```

Note: `expect` is dynamic `assert`; plain `assert` is static.

More generally: the concrete test case is concrete; we can write `assert`, in which case we need the method to have an `ensures` clause.

If we have `ensures` then we can verify the whole method under all inputs.

Back to proptests: this works

```
#[cfg(test)]
mod proptests {
 use super::*;
 use proptest::prelude::*;
 use proptest::num::u64;

 proptest! {
   #[test]
   fn stretched_rectangle_can_hold_original
      (width in u64::ANY, height in u64::ANY,
       factor in u64::ANY) {
     let original = Rectangle {
       width: width, height: height,
     };
     if let Some(larger) = original.stretch(factor) {
       assert!(larger.can_hold(&original));
     } } } }
```

All the way to Kani

```rust
// cargo kani --harness stretched_rectangle_can_hold_
#[cfg(kani)]
mod verification {
 use super::*;

 #[kani::proof]
 pub fn stretched_rectangle_can_hold_original() {
  let original = Rectangle {
      width: kani::any(),
      height: kani::any(),
  };
  let factor = kani::any();
  if let Some(larger) = original.stretch(factor) {
      assert!(larger.can_hold(&original));
  } } }
```

Proof harness implications

Kani verifies behaviour for *any* width, height, and factor.

It runs a whole bunch of tests, looking for safety violations (because Unsafe Rust).

But, also reports a failure!

```
Check 3: verification::stretched_rectangle_can_hold_original.asserti
  - Status: FAILURE
  - Description: "assertion failed: larger.can_hold(&original)"
  - Location: src/lib.rs:72:13 in function verification::stretched_r
[...]
SUMMARY:
 ** 1 of 40 failed
Failed Checks: assertion failed: larger.can_hold(&original)
 File: "src/lib.rs", line 72, in verification::stretched_rectangle_c

VERIFICATION:- FAILED
Verification Time: 0.08609796s
```

Why failure?

We're missing some preconditions.

We'll encode them directly into the harnesss:

```
kani::assume(0 != original.width);
kani::assume(0 != original.height);
kani::assume(1 < factor);
```

With these assumes (effectively preconditions), Kani verifies the code.

Analogous Dafny

```
method stretch(self: Rectangle, factor: nat) returns (rv : Rectangle)
    ensures rv.width = self.width * factor ∧
            rv.height = self.height * factor
{
    rv := new Rectangle;
    rv.width := self.width * factor;
    rv.height := self.height * factor;
    return rv;
}

method stretched_rectangle_can_hold_original
                (original: Rectangle, factor: nat)
    requires original.width > 0 ∧
             original.height > 0 ∧ factor > 1
{
    var larger := stretch(original, factor);
    assert can_hold(larger, original);
}
```

Part III

Dafny application: Cedar authorization-policy

Cedar: a use of Dafny in practice

At Amazon: Cedar authorization-policy language in Dafny.

Later: migrated to Lean, interactive theorem proving—more powerful.

https://www.amazon.science/blog/how-we-built-cedar-with-automated-reasoning-and-differential-testing

Cedar: write policies in domain-specific language;
execute queries on these policies;
response: “allow” or “deny”.

Cedar policy: TinyTodo policy 1 (allow)

```
// policy 1: enables access sometimes
permit(principal, action, resource)
when {
  resource has owner &&
  resource.owner == principal
};
```

principal is a User
resource is a List

Cedar policy: TinyTodo policy 3 (deny)

```
// policy 3
forbid (
  principal in Team::"interns",
  action == Action::"CreateList",
  resource == Application::"TinyTodo"
);
```

TinyTodo and Cedar

Idea: given policies in the DSL, TinyTodo doesn't contain reasoning about auth.

Just ask Cedar about whether an action is allowed or denied (and listen to it.)

Verification-guided development

Dafny (and Lean): prove properties of software.

Specifically: there are

1. explicit properties (specified by developers); and
2. implicit properties (must always be true).

Get formal guarantees—better than testing, if you can prove them.

Deployment and models

Usually don't want to deploy raw Dafny.

Fortunately, Dafny compiles to C#, Java, JavaScript, Go, and Python.

In the Cedar example: Dafny is a model of the Rust code.

Verifying the Dafny is still useful; it implies that a correct implementation is possible.

Using the Dafny

Dafny code, especially verified, can serve as an oracle.
We can look for divergences between Dafny & Rust.

Really, we want to know that the Rust code satisfies the properties.

Safe Rust: no undefined behaviour.
But what about contracts?

Assertions in Rust

Sure, but what if they fail?

Idea: differential random testing

Make sure that the Rust implementation matches the Dafny model.

Like fuzzing: generate millions of inputs, check equivalence.

Key properties of Cedar

Two overarching properties:

- explicit permit: permission granted only by individual *permit* policies, not gained by error or default;
- forbid overrides permit: any applicable *forbid* policy always denies access, even if *permit* policy allows it.

That is: authorization has to be explicitly granted by a specific policy; but if there is any forbid policy, then access is forbidden.

Example Request

principal=**User::"Alice"**
action=**Action::"GetList"**
resource=**List::"AliceList"**

Policy 1 yields **List::"AliceList" has owner && List::"AliceList".owner == User::"Alice"**

Say this is true. The request then *satisfies* the policy.

More generally, the authorization engine computes the request over all of its policies and returns an overall decision.

Buggy Dafny Code

```
function method isAuthorized(): Response { // BUGGY VERSION
  var f := forbids();
  var p := permits();
  if f ≠ {} then
    Response(Deny, f)
  else
    Response(Allow, p)
}
```

Why is this wrong with respect to earlier properties?

Patching with a predicate

```
// A request is explicitly permitted when a permit policy is satisfied
predicate IsExplicitlyPermitted(request: Request, store: Store) {
  ∃ p •
    p in store.policies.policies.Keys ∧
    store.policies.policies[p].effect = Permit ∧
    Authorizer(request, store).satisfied(p)
}

lemma AllowedIfExplicitlyPermitted(request: Request, store: Store)
ensures // A request is allowed if it is explicitly permitted
        (Authorizer(request, store).isAuthorized().decision = Allow) ⟹
        IsExplicitlyPermitted(request, store)
{...}
```

Trying to verify isAuthorized

Predicate `isExplicitlyPermitted`: there has to be a specific policy `p` which permits `request`.

Lemma `AllowedIfExplicitlyPermitted` needs to be shown—i.e. if authorizer says yes, then predicate `IsExplicitlyPermitted` has to be true.

When Dafny verifies `isAuthorized`, it tries to check that the lemma.

But the lemma doesn't hold, so Dafny will report "A postcondition might not hold on this return path", pointing at the ensures clause.

Fixing isAuthorized

```
// BUGGY VERSION
function method isAuthorized():
    Response {
var f := forbids();
var p := permits();
if f ≠ {} then
  Response(Deny, f)
else
  Response(Allow, p)
}
```

```
// FIXED VERSION
function method isAuthorized():
    Response {
  var f := forbids();
  var p := permits();
  if f = {} ∧ p ≠ {} then
    Response(Allow, p)
  else
    Response(Deny, f)
}
```

Dafny verifies the fixed version, and also the mirror property *forbid overrides permit*, which we don't show.

What Dafny shows

Other properties, notably that the Cedar validator is sound:

“if the validator accepts a policy, evaluating the policy should never result in certain classes of error.”

Making this proof go through enabled developers to fix subtle design bugs, which would otherwise be reflected in the implementation.

Usefulness of Dafny

We can use the specification for:

- automated reasoning, by Dafny; but also
- manual reasoning, by people.

Even good Rust is still an implementation language with lots of implementation details.

Reading the Dafny (or Lean) is much easier, since it has much less code (1/6th, at time of report).

Dafny and Rust

Are they the same?

If you don't know, bugs can sneak into the implementation.

Solution: differential random testing (DRT).

That is: randomly generate millions of inputs (access requests, data, and policies);
check that Dafny (now Lean) and Rust produce the same output.
If they disagree, that's a bug.

More on differential random testing

Inputs aren't completely randomly generated.
Same techniques as talked about in this class: input generators to create consistent inputs.
6 hours nightly of DRT, 100 million total tests.

Found errors in libraries (mostly parsers and IP address operations).

Cedar Summary

Cedar policy-authorization language.

Modelled in Dafny, showing key properties.

Rust implementation shown equivalent to Dafny using differential random testing.